SIMATS ENGINEERING
SAVEETHA SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI)

ASSIGNMENT 2 – CO2
DSA03 – NATURAL LANGUAGE PROCESSING

Build and Evaluate an N-gram Language Model and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline

Student Details	Information
Student Name	K VENKATA VAMSI GOUD
Register Number	192472021
Programme	B.Tech – CSE (AI)
Institution	SIMATS – Saveetha School of Engineering
Nature	Individual
Duration	1 Week
Total Marks	50
Submission Date	07-11-2026
Faculty Name	DR. Ram Kumar M

MARK SHEET (Rubric-based Evaluation)

Task	Title	Max Marks	Awarded
1	Counting Words and Vocabulary	3	
2	Unsmoothed N-gram Models	4	
3	Smoothing and Discounting	5	
4	Backoff and Deleted Interpolation	4	
5	Entropy, Cross-Entropy and Perplexity	4	
6	Word Classes and Tagset Design	4	
7	Rule-Based POS Tagging	4	
8	HMM Tagging and Viterbi	5	
9	Brill Tagging	4	
10	Implement and Test Models	3	
11	Compare and Recommend	2	
12	Day-4 Design Challenge	8	
	TOTAL	50	

MANDATORY SUBMISSION EVIDENCE CHECK

Task	Evidence expected	Status
1	Counting policy, Zipf plot, vocabulary thresholds and OOV	☐
2	Unigram/bigram/trigram probabilities and sparsity	☐
3	Add-one, add-k and Good–Turing	☐
4	Katz/stupid backoff, interpolation and deleted λ	☐
5	Entropy, perplexity and WER rescoring	☐
6	20–30 tags, Penn mapping and annotation agreement	☐
7	Tag dictionary and 15 contextual rules	☐
8	HMM estimates, Viterbi trellis and OOV method	☐
9	Brill templates and transformations	☐
10	Python implementation, tests, tables and plots	☐
11	Evidence-based recommendation	☐
12	Impact, evaluation, adaptation and defence	☐

Evidence note: the question requires 20,000 utterances / 150,000 tokens, 200 five-best ASR sets, 3,000 hand-tagged tokens and 500 double-annotated tokens. These datasets were not uploaded. Numerical worked examples are therefore illustrative; corpus-scale results must be calculated from your documented data. Do not present placeholders as measured results.

TASK 1 – COUNTING WORDS AND BUILDING THE VOCABULARY (3 MARKS)

1.1 Types, tokens, lemmas and word forms

Term	Definition	Example
Token	One occurrence.	“bus bus” = 2 tokens.
Type	Distinct item after normalisation.	“bus bus” = 1 type.
Word form	Observed inflected form.	book, books, booked.
Lemma	Dictionary base form.	books/booked → book, depending on POS.

1.2 Counting policy

Item	Policy	Reason
Case	Lowercase ordinary words; preserve raw transcript.	Vellore/vellore become one type.
Punctuation	Remove standalone punctuation for lexical counts; preserve internal apostrophes.	Punctuation is not a lexical word.
Numerals	Treat 6:30 as one time token and route 21 as one token.	Preserves timetable meaning.
Disfluencies	Retain uh/um separately; state whether excluded from lexical counts.	Useful speech evidence.
Boundaries	Add <s>, </s> for LM; exclude from word-type counts.	Models utterance edges.
Hyphenation	Keep known compound names together consistently.	Avoids split inconsistency.

Example: “Two tickets to Vellore at 6:30.” Case-folded lexical tokens are two | tickets | to | vellore | at | 6:30 (6). Splitting 6:30 into 6, colon and 30 produces 8 tokens. Preserving case makes Vellore and vellore separate types.

1.3 Corpus count report

Measure	Requirement / method	Value
Utterances	≥20,000	Fill from corpus
Word tokens	≥150,000	Fill from corpus
Word types	Unique normalised forms	Calculate
Lemmas	Unique POS-aware lemmas	Calculate
Word forms	Unique inflected forms	Calculate
Split	80/10/10 by utterance	Record exact counts

1.4 Zipf’s law and OOV

Zipf’s law: frequency f(r) is approximately C/r, where r is word rank. On log-log axes log f(r)≈log C−log r. Station names, route names, times, bus, train, ticket and fare terms may deviate due to domain frequency.

```
freq=Counter(tokens); ranked=sorted(freq.values(), reverse=True)

plt.loglog(range(1,len(ranked)+1),ranked,marker='.'); plt.xlabel('Rank'); plt.ylabel('Frequency')
```

Rare-word threshold	Vocabulary rule	UNK proportion
1	Keep types occurring ≥1	Calculate
2	Map frequency-1 types to <UNK>	Calculate
3	Map types with frequency <3	Calculate
5	Map types with frequency <5	Calculate

Development OOV rate = development tokens absent from training vocabulary / all development tokens. High OOV limits every n-gram model. Build vocabulary on training only and apply identical <UNK> handling to development/test.

TASK 2 – UNSMOOTHED N-GRAM MODELS (4 MARKS)

Chain rule: $P(W)=\Pi_i P(w_i|w_1 \ldots w_{i-1})$. Unigram assumes no history matters; bigram assumes only the immediately preceding word matters; trigram assumes only the previous two words matter.

Model	Maximum-likelihood estimate
Unigram	$P(w)=C(w)/N$
Bigram	$P(w_i\|w_{i-1})=C(w_{i-1},w_i)/C(w_{i-1})$
Trigram	$P(w_i\|w_{i-2},w_{i-1})=C(w_{i-2},w_{i-1},w_i)/C(w_{i-2},w_{i-1})$

Illustrative training utterances: <s> two tickets to vellore </s>; <s> two tickets to chennai </s>; <s> one ticket to vellore </s>. Counts: C(two)=2, C(tickets)=2, C(two,tickets)=2, C(tickets,to)=2, C(to,vellore)=2, C(two,tickets,to)=2, C(tickets,to,vellore)=1.

For “two tickets to vellore”, unigram probability multiplies C(w)/N for each token. Bigram probability is P(two|<s>)×P(tickets|two)×P(to|tickets)×P(vellore|to)×P(</s>|vellore). Trigram probability uses two-word histories. Substitute actual corpus counts for final numeric values.

Zero-probability example: “two tickets to Katpadi” is grammatical, but if (to, katpadi) is absent then P(katpadi|to)=0 and the complete unsmoothed bigram sentence probability is zero.

Bigram sparsity=1−observed distinct bigrams/$|V|^2$; trigram sparsity=1−observed distinct trigrams/$|V|^3$. Development unseen rate=unseen n-gram occurrences/all development n-gram occurrences. Unigram generation has no word order; bigram improves local order; trigram improves context but increases sparsity and data needs.

TASK 3 – SMOOTHING AND DISCOUNTING (5 MARKS)

Zero count is not proof that a sequence is impossible; it only means it was absent from finite training data. A valid conditional distribution requires $P(w|h) \geq 0$ and $\Sigma w \in V\ P(w|h) = 1$.

Method	Formula	Effect
Add-one	$(C(h,w)+1)/(C(h)+V)$	One pseudo-count per vocabulary word.
Add-k	$(C(h,w)+k)/(C(h)+kV)$	Tunable pseudo-count.

Illustration: context "to" has counts vellore=2, chennai=1, katpadi=0; assume V=10 and C(to)=3. Add-one gives 3/13, 2/13 and 1/13 respectively; each other vocabulary item also receives 1/13. Add-k with k=.1 gives 2.1/4, 1.1/4 and .1/4. Large V makes add-one over-discount because the denominator C(h)+V grows even when most words are implausible continuations.

Good–Turing: let N_r be the number of n-gram types occurring r times. Adjusted count $c^* = (r+1)N_{r+1}/N_r$. Approximate unseen probability mass is N_1/N. At high counts N_{r+1} may be zero or unstable; smooth the frequency-of-frequencies or use original counts above a threshold.

r	Illustrative N_r	Adjusted count
1	10	$2N_2/N_1$
2	6	$3N_3/N_2$
3	3	$4N_4/N_3$
4	1	$5N_5/N_4$

Compare add-one, add-k and Good–Turing on development perplexity and the probability assigned to an unseen bigram. Insert actual N_r counts and measured perplexities from your corpus.

TASK 4 – BACKOFF AND DELETED INTERPOLATION (4 MARKS)

Method	Definition
Backoff	Use highest-order model available; consult shorter context only if longer one is unseen.
Interpolation	Weighted sum of unigram, bigram and trigram probabilities; all orders contribute.

Katz backoff discounts observed n-grams and assigns leftover mass to unseen events through a lower-order model: $P(w|h)=\alpha(h)P_lower(w)$ for an unseen event. $\alpha(h)$ normalises the distribution. Stupid backoff uses a fixed multiplier (often .4) when backing off; it produces a ranking score, not a probability, because it is not normalised.

Linear interpolation: $P=\lambda_3 P_trigram+\lambda_2 P_bigram+\lambda_1 P_unigram$, where $\lambda_1+\lambda_2+\lambda_3=1$. Deleted interpolation estimates weights by deleting each event from counts, comparing leave-one-out estimates at each order and assigning evidence to the best order.

Context count	λ unigram	λ bigram	λ trigram	Reason
Low 1–2	0.55	0.35	0.10	Lower orders reliable.
Medium 3–10	0.25	0.50	0.25	Bigram context useful.
High >10	0.10	0.30	0.60	More evidence for trigram.

Weights above are illustrative only; estimate actual weights on the specified held-out training portion. For unseen “tickets to Katpadi”, Katz backs off and applies α; interpolation uses $\lambda_3\times 0+\lambda_2 P(Katpadi|to)+\lambda_1 P(Katpadi)$.

TASK 5 – ENTROPY, CROSS-ENTROPY AND PERPLEXITY (4 MARKS)

Measure	Formula	Meaning
Entropy	$H(P)=-\Sigma P(x)\log_2 P(x)$	Uncertainty.
Cross-entropy	$H(P,Q)=-\Sigma P(x)\log_2 Q(x)$	Average coding cost under Q.
Perplexity	$PP=2^H$	Effective number of equally likely next words.

Perplexity 90 means average uncertainty equivalent to choosing among 90 equally likely next-word options. Compare only models with identical test data, vocabulary and <UNK> policy.

Hand calculations: uniform P=(.25,.25,.25,.25) gives H=2 bits. P=(.5,.3,.2) gives H≈1.485 bits. Sharply peaked P=(.9,.0333,.0333,.0334) gives approximately .63 bits.

Held-out cross-entropy $H=-(1/N)\Sigma\log_2 P(w_i|history_i)$; $PP=2^H$. Report word-average and per-utterance negative log likelihood.

Model	Word H	Word PP	Utterance NLL	WER
Unigram	Measure	2^H	Measure	Measure
Unsmoothed bigram	Measure / ∞ if zero	2^H / ∞	Measure	Measure
Unsmoothed trigram	Measure / ∞ if zero	2^H / ∞	Measure	Measure
Add-k / Good–Turing	Measure	2^H	Measure	Measure
Katz / interpolation	Measure	2^H	Measure	Measure

Different vocabularies or <UNK> thresholds make perplexity comparisons unfair: aggressive unknown mapping collapses difficult words. For 200 five-best ASR sets, combine LM log probability with acoustic score, select the best hypothesis and compute WER=(S+D+I)/reference words×100. Lower perplexity need not lower WER if a fluent but wrong station name is favoured.

TASK 6 – ENGLISH WORD CLASSES AND TAGSET DESIGN (4 MARKS)

Class	Open/closed	Examples
Nouns	Open	ticket, bus, Vellore, station, fare
Verbs	Open	book, travel, cancel, return
Adjectives	Open	available, delayed, express
Adverbs	Open	early, late, directly
Determiners	Closed	a, the, this
Pronouns	Closed	I, you, it
Prepositions	Closed	to, from, at, via
Conjunctions	Closed	and, but, or
Auxiliaries/modals	Closed	is, have, can, will
Interjections/particles	Mostly closed	uh, um, up, off

Closed classes provide syntactic structure and occur frequently. Mis-tagging prepositions can alter origin/destination interpretation.

Ambiguous word	Tags	Distributional examples
book	NN/VB	a book / book a ticket
return	NN/VB/JJ	a return / return tomorrow / return ticket
stand	NN/VB	bus stand / please stand
train	NN/VB	the train / train staff
fast	JJ/RB	fast train / run fast

Tag	Domain meaning	Penn mapping
NN	Common noun	NN
NNS	Plural noun	NNS
NNP	Proper noun/station/place	NNP
VB	Base verb	VB
VBD	Past verb	VBD
VBG	Gerund/participle	VBG
VBN	Past participle	VBN
VBZ	3rd-person singular	VBZ
VBP	Non-3rd-person present	VBP
JJ	Adjective	JJ
JJR	Comparative adjective	JJR
JJS	Superlative adjective	JJS
RB	Adverb	RB
RBR	Comparative adverb	RBR
RBS	Superlative adverb	RBS
DT	Determiner	DT
PRP	Personal pronoun	PRP
PRP$	Possessive pronoun	PRP$
IN	Preposition/subordinator	IN
CC	Coordinating conjunction	CC
MD	Modal	MD
TO	To marker	TO/IN by context
CD	Number/time/fare	CD
UH	Interjection/disfluency	UH
RP	Particle	RP
SYM	Currency/symbol	SYM
FW	Code-mixed/foreign	FW

This 27-tag inventory covers open and closed classes and adds FW for code-mixed tokens. Separate tags for every station subtype or fare category are unnecessary; those are better represented as entity/slot attributes. For 3,000 annotated tokens and 500 double-annotated tokens, raw agreement=matching tags/joint tokens×100; Cohen's $\kappa=(P_o-P_e)/(1-P_e)$. Report tag frequencies, ambiguous type/token rates and average tags per ambiguous token from the annotation.

TASK 7 – RULE-BASED POS TAGGING (4 MARKS)

Two stages: (1) dictionary assigns all possible tags to each word; (2) contextual constraints remove impossible tags. Example: book may be NN/VB; after DT choose NN, after MD choose VB.

Rule	Condition	Action
R1	Previous tag DT	Prefer NN/NNS
R2	Previous tag MD	Choose VB
R3	Previous tag TO and verb candidate	Choose VB
R4	Previous tag PRP	Choose VBP/VBZ by agreement
R5	-ing after form of be	Choose VBG
R6	-ed after have/has/had	Choose VBN
R7	Suffix -ly	Prefer RB
R8	Suffix -tion/-ment/-ness	Prefer NN
R9	Suffix -ous/-ive/-able	Prefer JJ
R10	Capitalised non-sentence-initial token	Prefer NNP
R11	Time/number pattern	Choose CD
R12	Previous tag JJ	Prefer NN
R13	Following noun; token return/express	Prefer JJ modifier
R14	Previous tag IN/TO and place name	Choose NNP
R15	Token uh/um/er	Choose UH

Conflict: sentence-initial capitalisation can cause a common noun to be tagged NNP. Suppress that rule at sentence start unless a gazetteer confirms a place name. Report dictionary coverage, candidate ambiguity, overall accuracy, ambiguous-token accuracy and five common confusion pairs from the test confusion matrix.

TASK 8 – STOCHASTIC HMM TAGGING AND VITERBI (5 MARKS)

Bayes: $P(T|W) \propto P(W|T)P(T)$. HMM assumptions: each word depends only on its current tag (emission independence); each tag depends only on previous tag (first-order Markov). Best sequence maximises $\Pi_i P(w_i|t_i)P(t_i|t_{i-1})$.

Probability	MLE	Additive smoothing
Transition	C(prev,current)/C(prev)	(C(prev,current)+α)/(C(prev)+α|T|)
Emission	C(tag,word)/C(tag)	Add-β or OOV class model

After smoothing, each transition row must sum to 1.000. Illustrative sentence “book return ticket to Vellore”: candidates book=VB/NN; return=NN/VB/JJ; ticket=NN; to=TO/IN; Vellore=NNP. A likely command-context path is VB NN NN TO NNP.

Position	Word	Candidate tags	Illustrative best	Backpointer
1	book	VB/NN	VB	START
2	return	NN/VB/JJ	NN	VB
3	ticket	NN	NN	NN
4	to	TO/IN	TO	NN
5	Vellore	NNP	NNP	TO

Viterbi recurrence $V_i(t)=\max_{t'}[V_{i-1}(t')P(t|t')P(w_i|t)]$. Store predecessor $B_i(t)$, then backtrack from best final state. The table is a structural illustration; insert numeric probabilities into every trellis cell using your annotated training corpus.

Method	Operations	Comment
Most-frequent tag	O(n)	Fast, ignores context.
Exhaustive sequences	$O(K^n)$	Exact but exponential.
Viterbi	$O(nK^2)$	Dynamic programming.

For unknown words use suffix clues (-ing→VBG, -ed→VBD/VBN, -ly→RB), capitalisation (NNP), numeric patterns (CD) and open-class priors. Report OOV-token accuracy separately.

TASK 9 – TRANSFORMATION-BASED (BRILL) TAGGING (4 MARKS)

Brill starts from an initial tagger (often most-frequent-tag). Templates generate candidate corrections; greedily select the rule correcting the most errors minus introduced errors. Stop when no rule exceeds minimum gain or max rule count.

Template	Example
Previous tag	NN→VB if previous tag MD
Next tag	NN→VB if next tag TO
Previous word	NN→VB if previous word please
Next word	NN→JJ if next word ticket and token return
Previous two tags	Use determiner + adjective context
Suffix	Unknown word ending -ing → VBG
Capitalisation	NN→NNP for gazetteer station
Specific word	Correct frequent lexical ambiguity

Order	Plausible rule candidate	Corrected	Introduced
1	NN→VB after MD	Measure	Measure
2	NN→NNP for known station	Measure	Measure
3	NN→VB after please	Measure	Measure
4	NN→VBG for -ing after VBZ	Measure	Measure
5	VBD→VBN after have/has/had	Measure	Measure
6	NN→CD for numeric/time	Measure	Measure
7	VB→NN after DT	Measure	Measure
8	JJ→RB for -ly after verb	Measure	Measure
9	NN→TO for token to	Measure	Measure
10	NN→UH for uh/um	Measure	Measure

These are candidate examples, not claimed learned rules. Replace with the actual first ten learned transformations and their corrected/introduced counts. A general modal + base verb rule is linguistic; a one-off training typo rule is an artefact. Plot accuracy against number of transformations and stop at the development-set peak.

TASK 10 – IMPLEMENT AND TEST THE MODELS (3 MARKS)

Core implementation outline (Python standard library; NLTK may be used for standard tagger/stemmer utilities):

```
import re, math, random

from collections import Counter

random.seed(42)

def tokenise(text): return re.findall(r'\\d+:\\d+|[a-z]+(?:\'[a-z]+)?|\\d+', text.lower())

def counts(sents):

    U,B,T=Counter(),Counter(),Counter()

    for sent in sents:

        s=['<s>','<s>']+sent+['</s>']; U.update(s)

        B.update(zip(s[:-1],s[1:])); T.update(zip(s[:-2],s[1:-1],s[2:]))

    return U,B,T

def add_k(w,h,B,U,V,k=.1): return (B[(h,w)]+k)/(U[h]+k*V)

def viterbi(words,tags,trans,emit,unk):

    V=[]; back=[]

    for i,w in enumerate(words):

        cur={}; bp={}

        for t in tags:

            e=emit.get((t,w),unk.get(t,1e-8))

            if i==0: cur[t]=trans.get(('<s>',t),1e-8)*e; bp[t]='<s>'

            else:

                best,prev=max((V[-1][p]*trans.get((p,t),1e-8)*e,p) for p in tags)

                cur[t]=best; bp[t]=prev

        V.append(cur); back.append(bp)

    last=max(V[-1],key=V[-1].get); seq=[last]

    for i in range(len(words)-1,0,-1): seq.append(back[i][seq[-1]])

    return list(reversed(seq)),max(V[-1].values())
```

Step	Action	Evidence
1	Document source, cleaning and seed 42	README/manifest
2	Split utterances 80/10/10 before tuning	Split IDs
3	Build vocabulary from training only	UNK threshold table
4	Train all n-gram/smoothing models	Counts/parameters
5	Tune k and λ on development	Configuration
6	Train POS taggers	Dictionary/rules/HMM
7	Evaluate once on test	PP/WER/accuracy/OOV
8	Repeat timing; report median	Runtime/memory log

Model	Training time	Memory	Held-out metric	Speed
Unigram/bigram/trigram	Measure	Measure	PP or accuracy	Tokens/sec

Add-k/Good–Turing	Measure	Measure	PP or accuracy	Tokens/sec
Katz/interpolation	Measure	Measure	PP or accuracy	Tokens/sec
Rule POS	Measure	Measure	PP or accuracy	Tokens/sec
HMM POS	Measure	Measure	PP or accuracy	Tokens/sec
Brill POS	Measure	Measure	PP or accuracy	Tokens/sec

Required plots: (1) perplexity against n-gram order/smoothing; (2) POS accuracy against training-set size or Brill transformation count. Verify zero-probability example, λ weights, Viterbi trellis and initial Brill rules against hand calculations.

TASK 11 – COMPARE, RECOMMEND AND DEFEND (2 MARKS)

Criterion	Unsmoothed LM	Smoothed LM	Rule POS	HMM POS	Brill POS
Coverage	Poor on unseen	Handles unseen	Dictionary-limited	Needs OOV method	Depends on initial tagger
Accuracy	Sparse weakness	Measure	Transparent rules	Statistical context	Corrects recurring errors
Speed	Fast	Fast after training	Fast	$O(nK^2)$	Fast at inference
Maintenance	Low	Tune weights	Manual rules	Needs labels	Templates + labels

Provisional choice: Good–Turing/Katz or deleted interpolation selected by held-out perplexity and ASR WER, rather than unsmoothed counts. Use HMM/Viterbi as statistical POS baseline and add Brill corrections only if development evaluation shows improvement. Consider OOV accuracy, speed and maintenance, not accuracy alone. A caller experiences a wrong destination, timetable, fare or refund interpretation; proper-noun recognition is critical.

TASK 12 – DAY-4 DESIGN CHALLENGE (8 MARKS)

12A. Impact analysis (2 marks)

Metro and intercity rail add station/train names and fare categories. Code-mixed speech and phonetic spelling variants increase; over 15% of tokens are unseen. Stable vocabulary, English-only word classes and old-domain transition estimates are no longer valid.

Change	Broken assumption	Impact
New station names	Stable vocabulary	OOV; destination words get low/zero LM score.
Code-mixing	English-only tagset	Tokens unrecognised or mistagged.
Phonetic variants	Stable spelling	One station has multiple forms.
New fare terms	Old-domain distribution	Transition/emission mismatch.
OOV >15%	Low stable OOV	Perplexity and tagging errors rise.

Place names are the most damaging OOV tokens because they often carry the destination slot. Losing one can make the enquiry unusable.

12B. Evaluation of original design (2 marks)

New utterance	Likely failure	First affected component
Next train to Arakkonam	Unseen station; LM maps to UNK; tagger may miss NNP.	Vocabulary/LM and proper noun tag.
Chennai to Madurai bus timing enna	English LM has no context for enna; no code-mix tag.	Tokeniser, language ID, tagset.
Chennai to Maturai train time sollunga	Phonetic variant unseen; code-mixed words OOV.	OOV and gazetteer.

Measure new-domain perplexity, WER after rescoring and POS accuracy against original baselines. LM and proper-noun/OOV tagging should degrade fastest. Do not invent numerical degradation; annotate and test a new sample.

12C. Adaptation or redesign (2 marks)

Decision: MODIFY. Retain n-gram sequence scoring but add open-vocabulary support and domain adaptation.

Component	Change	Benefit
Vocabulary	Refresh station/train/fare gazetteers; character/subword model.	New names and variants.
LM	Class-based n-grams for station, train, time, fare.	Shares context evidence.
Smoothing	Re-estimate Good–Turing and deleted-interpolation λ on mixed data.	Adapts probability mass.
Tagset	Keep 27 tags; consistently use FW and NNP; add language-ID attribute.	Represents code-mixing.
Tagger	HMM with character/suffix OOV features and gazetteer priors; optional Brill.	Better unseen proper nouns.
Refresh	Monitor OOV/WER weekly; update gazetteer monthly or on drift.	Current coverage.

12D. Evidence-based defence (2 marks)

Modification is preferable to retaining the old closed-vocabulary system; full replacement is unnecessary because n-gram/HMM methods remain efficient. Gazetteers preserve station identity, class-based n-grams share evidence, character models handle spelling variants and FW represents code-mixed tokens.

Evaluate on a separately held-out mixed-domain set: perplexity, WER, OOV tagging accuracy, NNP accuracy, overall POS accuracy, speed and maintenance. Trigger refresh if OOV exceeds 15% or destination accuracy falls below the service threshold.

A fixed-order n-gram only conditions on a finite preceding context. It cannot capture arbitrary long dependencies, discourse, speaker intention or world knowledge simply by receiving more data. Keep human confirmation for low-confidence destinations/fares.

Final recommendation

Deploy a modified hybrid: interpolated/backoff LM with refreshed counts, class-based and character-level station-name support, station/train gazetteer, and HMM POS tagging with OOV features plus optional Brill corrections. Preserve original transcript and confidence; ask confirmation when destination/service is uncertain.

APPENDIX – FINAL SUBMISSION CHECKLIST AND FORMULAS

Requirement	Check
Corpus	☐ 20,000+ utterances; 150,000+ tokens.
Splits	☐ 80/10/10; no leakage; fixed seed.
ASR	☐ 200 five-best sets with references.
Annotation	☐ 3,000 tokens; 500 double-tagged; κ.
Models	☐ All n-gram, smoothing, backoff, interpolation and taggers.
Evaluation	☐ Held-out perplexity, WER, tagging accuracy.
Plots	☐ Zipf, perplexity comparison, tagging curve.
Challenge	☐ Three mixed-domain examples and measured degradation.
Submission	☐ Replace placeholders with actual results and screenshots.

Concept	Formula
Unigram	$P(w)=C(w)/N$
Bigram	$P(w_i\|w_{i-1})=C(w_{i-1},w_i)/C(w_{i-1})$
Trigram	$P(w_i\|w_{i-2},w_{i-1})=C(w_{i-2},w_{i-1},w_i)/C(w_{i-2},w_{i-1})$
Add-k	$(C(h,w)+k)/(C(h)+kV)$
Good–Turing	$c^*=(r+1)N_{r+1}/N_r$
Cross-entropy	$H=-(1/N)\Sigma\log_2 P(w_i\|history_i)$
Perplexity	$PP=2^H$
WER	$(S+D+I)/N\times100$
Cohen's κ	$(P_o-P_e)/(1-P_e)$
Viterbi	$V_i(t)=\max_{t'}V_{i-1}(t')P(t\|t')P(w_i\|t)$

End of Assignment 2 – CO2.